

Assignment 5 - N-Body

M11315016 - 林品安

1 Implementation

1.1 What is your parallelism strategy?

The current implementation employs several parallelization strategies:

1. Thread-level Parallelism:

```
1 static constexpr int BLOCK_SIZE = 256;
2 // Each thread processes one particle's interactions
3 int i = blockIdx.x * blockDim.x + threadIdx.x;
```

2. Tiled Computation with Shared Memory:

```
1 extern __shared__ double shmem[];
2 // Shared memory layout optimization
3 double* sh_qx = shmem;
4 double* sh_qy = shmem + BLOCK_SIZE;
5 double* sh_qz = shmem + 2 * BLOCK_SIZE;
```

3. **Pipeline Structure:** - Compute accelerations kernel - Update positions and velocities kernel - Collision detection kernel

1.2 Optimization techniques?

1. Memory Access Optimization:

```
1 // Coalesced memory access pattern
2 double xi = qx[i];
3 double yi = qy[i];
4 double zi = qz[i];
```

2. Computational Optimization:

```
1 // Fast math operations
2 double invDist = rsqrt(dist2);
3 double invDist3 = invDist * invDist * invDist;
```

3. Loop Unrolling:

```
1 #pragma unroll 8
2 for (int j = 0; j < BLOCK_SIZE; j++) {
3     // Force computation
4 }
```

4. **Shared Memory Usage:** - Reduces global memory bandwidth - Enables data reuse within thread blocks
- Optimized layout for minimal bank conflicts

5. Memory Transfer Optimization:

```
1 size_t shared_mem_size = (4 * BLOCK_SIZE * sizeof(double))
2                          + (BLOCK_SIZE * sizeof(char));
```

1.3 How do you manage the 2-GPU resources?

I did try the 2-GPU method, but the implementation faces some issues due to a bottleneck in communication between the GPUs. To effectively manage 2 GPUs, I would implement the following approach:

```

1 class MultiGPUSimulation {
2     struct GPUContext {
3         int device_id;
4         hipStream_t stream;
5         double* d_data;
6     };
7     vector<GPUContext> contexts;
8
9     void initialize() {
10         for(int i = 0; i < 2; i++) {
11             hipSetDevice(i);
12             GPUContext ctx;
13             ctx.device_id = i;
14             hipStreamCreate(&ctx.stream);
15             contexts.push_back(ctx);
16         }
17     }
18 };

```

2 If there are 4 GPUs instead of 2, what would you do to maximize the performance?

When increasing from 2 GPUs to 4 GPUs, the approach to parallelization needs to be adjusted to maximize performance. The following strategies could help achieve better scaling with 4 GPUs:

1. **Spatial Decomposition:** The domain can be divided into 4 smaller regions, each assigned to a GPU. This ensures that the workload is distributed evenly among the GPUs. Each region of space can handle a portion of the particles, reducing the load on each GPU.

```

1 struct Domain {
2     Vec3 min_bound, max_bound;
3     vector<int> particle_indices;
4 };
5
6 vector<Domain> decompose_space(int num_gpus) {
7     vector<Domain> domains(num_gpus);
8     // Divide space into 4 regions (for 4 GPUs)
9     return domains;
10 }

```

2. **Load Balancing:** Load balancing between the GPUs is essential to avoid underutilization of resources. It can be achieved by ensuring that the number of particles assigned to each domain is as even as possible.

```

1 void balance_load(vector<Domain>& domains) {
2     int total_particles = 0;
3     for(auto& domain : domains) {
4         total_particles += domain.particle_indices.size();
5     }
6     int target_per_gpu = total_particles / domains.size();
7     // Adjust domain boundaries to balance the workload
8 }

```

3. **Communication Optimization:** As the number of GPUs increases, inter-GPU communication becomes a bottleneck. To mitigate this, boundary data (particles on the edges of domains) must be exchanged efficiently between GPUs. This requires minimal overhead in terms of synchronization and data transfer.

```

1 void exchange_boundary_data() {
2     for(int i = 0; i < num_gpus; i++) {
3         for(int j = 0; j < num_gpus; j++) {
4             if(domains[i].touches(domains[j])) {
5                 transfer_boundary_particles(i, j);
6             }
7         }
8     }
9 }

```

In summary, the key to maximizing performance with 4 GPUs involves efficient spatial decomposition, balanced workload distribution, and optimized communication between the GPUs to reduce bottlenecks and improve scaling.

3 If there are 5 gravity devices, is it necessary to simulate 5 n-body simulations independently for this problem?

No, it is not necessary to simulate 5 independent N-body simulations. Rather, a single simulation can be optimized by using shared computations and state tracking to account for the presence of multiple gravity devices. Here's how:

1. **State Tracking Approach:** Instead of running separate simulations, could track the states of the 5 gravity devices and their interactions with the particles. This can be done using a state structure that updates the properties of each device as the simulation progresses.

```
1  struct SimulationState {
2      vector<bool> active_devices;
3      vector<double> device_masses;
4
5      void update_state(int step) {
6          for(int i = 0; i < active_devices.size(); i++) {
7              if(active_devices[i]) {
8                  device_masses[i] = compute_mass(i, step);
9              }
10         }
11     }
12 };
```

2. **Efficient Mass Updates:** Instead of simulating each gravity device independently, could update the mass of each device during the simulation step, ensuring that interactions are correctly accounted for without duplicating the N-body simulation.

```
1  void simulate_with_devices() {
2      uint32_t device_mask = 0x1F; // All 5 devices active
3      while(device_mask > 0) {
4          update_masses(device_mask);
5          simulate_step();
6          device_mask = update_device_states(device_mask);
7      }
8  }
```

3. **Optimization Benefits:** By using a shared simulation framework, could be benefit from below

- Single memory allocation for all particles and devices
- Shared force computations (reducing redundant calculations)
- Reduced kernel launches for multiple devices
- Better cache utilization, since all data is in the same simulation context
- Minimal synchronization overhead, as all devices are managed within the same simulation

This approach reduces the computational burden, as running 5 separate simulations would be inefficient and redundant. By efficiently managing the gravity devices in a single simulation, could maintain optimal performance while accurately modeling the interactions between particles and devices.